Engenharia de Software – Aula 25

Sprint 04: Persistência Relacional com SQLite e Padrão Repository

Prof. Me. Reginaldo Pereira Fernandes

Instituto Federal de Educação, Ciência e Tecnologia do Ceará – Campus Tauá
Técnico em Informática para Internet (TSII.210)

Semana 1 – Encontro 2 (100 min)

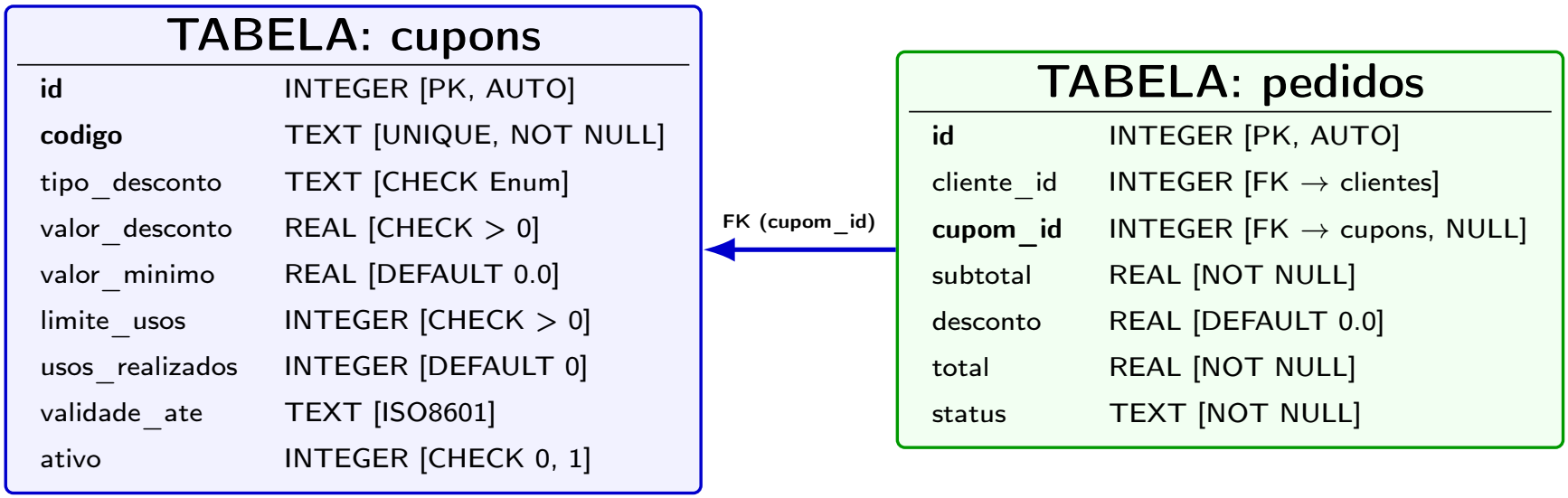
Agenda do Encontro 2 (100 Minutos)

1. Mapeamento Objeto-Relacional e DER
2. Arquitetura em Camadas e Repositório
3. Implementação: DDL SQLite
4. Implementação: Repositório CupomRepository
5. Rito Ágil e Próximos Passos

Do Modelo de Classes à Persistência em Banco

- **Status da Sprint 04:**
  - Na Aula 24, modelamos os Casos de Uso e Classes e implementamos `exceptions.py` e a classe `Cupom`.
- **O Problema da Memória Volátil:**
  - Objetos em memória RAM desaparecem quando o processo encerra.
  - Os cupons da livraria universitária precisam persistir em disco no banco **SQLite**!
- **Mapeamento Objeto-Relacional (ORM):**
  - Como transformar instâncias de `Cupom` em registros relacionais?
  - Como proteger as regras de negócio em duas camadas: código Python E restrições SQL?

Diagrama Entidade-Relacionamento: Esquema Relacional (DER)



Associação 0..1: Um pedido pode ou não conter um cupom aplicado.

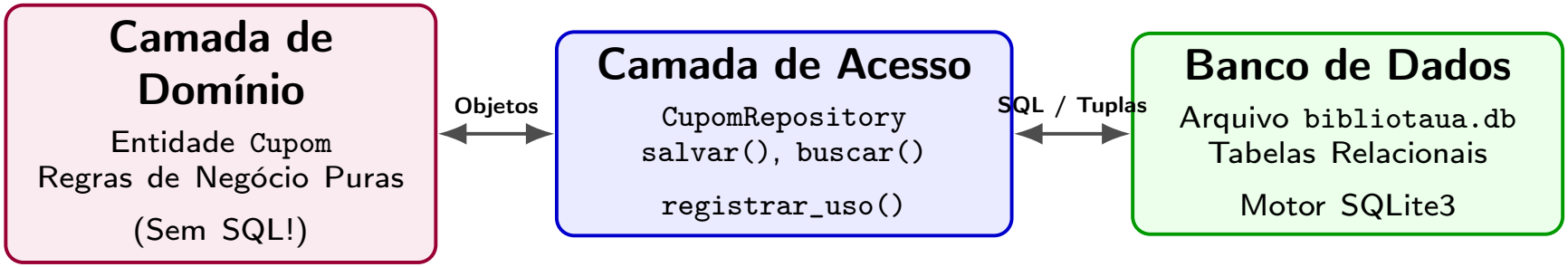
Restrições de Integridade (Constraints) no SQLite

Defesa em Profundidade: Por que validar no Banco de Dados?

Mesmo que um desenvolvedor esqueça de validar no Python, o motor do SQLite rejeitará dados corrompidos!

- `UNIQUE(codigo)`: Impede duplicidade de nomes de cupons na livraria.
- `CHECK(tipo_desconto IN ('PERCENTUAL', 'VALOR_FIXO'))`: Garante consistência com o Enum em Python.
- `CHECK(valor_desconto > 0)`: Proíbe descontos nulos ou negativos.
- `FOREIGN KEY(cupom_id) REFERENCES cupons(id)`: Integridade referencial garantida.

Diagrama de Arquitetura: Padrão Repository



Princípio de Responsabilidade Única (SRP)

A entidade `Cupom` não sabe se os dados vieram de um banco SQLite, PostgreSQL ou arquivo JSON. Toda a persistência é encapsulada pelo repositório!

Blindagem contra SQL Injection (Regra Inegociável)

Vulnerabilidade Crítica

Concatenação de Strings:

```
# NUNCA FACA ISSO!
query = f"SELECT * FROM cupons WHERE
    codigo = '{cod}'"
cursor.execute(query)
# Permite: ' OR 1=1; DROP TABLE...
```

Padrão Seguro (Parametrizado)

Parâmetros Posicionais:

```
# SEGURO: Parametros com ?
query = "SELECT * FROM cupons WHERE
    codigo = ?"
cursor.execute(query, (cod,))
# O driver trata caracteres especiais
    como dado puro!
```

Mão na Massa: DDL da Tabela cupons (database.py)

```
CREATE TABLE IF NOT EXISTS cupons (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    codigo TEXT NOT NULL UNIQUE,
    tipo_desconto TEXT NOT NULL CHECK(tipo_desconto IN ('PERCENTUAL', '
        VALOR_FIXO')),
    valor_desconto REAL NOT NULL CHECK(valor_desconto > 0),
    valor_minimo_pedido REAL NOT NULL DEFAULT 0.0 CHECK(valor_minimo_pedido
        >= 0),
    limite_usos INTEGER NOT NULL DEFAULT 100 CHECK(limite_usos > 0),
    usos_realizados INTEGER NOT NULL DEFAULT 0 CHECK(usos_realizados >= 0),
    validade_ate TEXT NOT NULL,
    ativo INTEGER NOT NULL DEFAULT 1 CHECK(ativo IN (0, 1))
);

CREATE INDEX IF NOT EXISTS idx_cupons_codigo ON cupons(codigo);
```

Mão na Massa: CupomRepository (Persistência e Busca)

```
class CupomRepository:
    def __init__(self, conn: sqlite3.Connection):
        self.conn = conn

    def salvar(self, cupom: Cupom) -> Cupom:
        cursor = self.conn.cursor()
        cursor.execute("""
            INSERT INTO cupons (codigo, tipo_desconto, valor_desconto,
                valor_minimo_pedido, limite_usos, validade_ate, ativo)
            VALUES (?, ?, ?, ?, ?, ?, ?)
        """, (cupom.codigo, cupom.tipo_desconto.value, cupom.valor_desconto,
              cupom.valor_minimo_pedido, cupom.limite_usos, cupom.
                  validade_ate.isoformat(), 1 if cupom.ativo else 0))
        self.conn.commit()
        cupom.id = cursor.lastrowid
        return cupom

    def buscar_por_codigo(self, codigo: str) -> Optional[Cupom]:
        cursor = self.conn.cursor()
        cursor.execute("SELECT * FROM cupons WHERE codigo = ?", (codigo.strip
            ().upper(),))
        row = cursor.fetchone()
        return self._row_para_objeto(row) if row else None
```

Mão na Massa: Incremento Atômico Seguro (registrar_uso)

```
    def registrar_uso(self, cupom_id: int) -> bool:
        """Incrementa atomicamente os usos realizados sem risco de corrida."""
        cursor = self.conn.cursor()
        cursor.execute("""
            UPDATE cupons
            SET usos_realizados = usos_realizados + 1
            WHERE id = ? AND usos_realizados < limite_usos AND ativo = 1
        """, (cupom_id,))
        self.conn.commit()
        if cursor.rowcount == 0:
            raise CupomEsgotadoError(0)
        return True
```

Atomicidade e Proteção contra Concorrência

A cláusula `WHERE usos_realizados < limite_usos` impede que dois clientes utilizem o último cupom simultaneamente!

Rito Ágil: Registro da Daily 2

Anotação para o Relatório da Sprint no `guia_desenvolvedor.md`

- **O que realizei hoje?**
  - Projetei o Diagrama Entidade-Relacionamento (DER) integrando `cupons` e `pedidos`.
  - Escrevi o script DDL com restrições `CHECK`, `UNIQUE` e índices em `database.py`.
  - Implementei a classe `CupomRepository` com queries parametrizadas blindadas.
  - Implementei o incremento atômico de utilizações `registrar_uso`.
- **O que farei no Encontro 3 (Aula 26)?**
  - Analisar o Diagrama de Sequência do checkout.
  - Implementar o serviço de cálculo `CalculadoraDescontoService`.
  - Integrar o cupom ao agregado `Pedido` e implementar o `CheckoutService`.
- **Impedimentos?** Nenhum impedimento.

Síntese do Encontro 2

- **Conquistas de Hoje:**
  - Mapeamento ORM concluído com sucesso.
  - Persistência relacional protegida em SQLite.
  - Camada de dados isolada da camada de regras de negócio.
- **Para o Encontro 3 (Aula 26):**
  - Orquestração do Negócio: **Diagrama de Sequência UML**.
  - Cálculo de desconto e finalização de pedidos.
  - Baixa no estoque de livros e transações completas.

Excelente progresso! A persistência do BiblioTauá está blindada!